UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Computer generations and history

Source: CSC429 (8).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The machine of the zero computer generation is called a mechanical computer because it operated entirely by physical/mechanical means — gears, levers, wheels, rods and shafts — rather than by electronic components. It had no electronic circuitry, no stored-program memory and no electronic switching; all computation was performed through the movement and positioning of mechanical parts, with the operator setting values and reading results physically. Hence it is a 'mechanical' (not electronic) computer. [1 mark]

(b) Completed table:

	Mechanical Calculating Machine	Difference Machine
Inventor	Blaise Pascal (Pascualine/Pascaline, 1642) — alternatively Wilhelm Schickard's calculating clock; Gottfried Leibniz's stepped reckoner is a later mechanical calculator	Charles Babbage (Difference Engine, c. 1822)
Function	Performs basic arithmetic (addition and subtraction, with later machines such as Leibniz's also multiplication and division) on numbers entered by the operator, mechanically, to aid calculation	Automatically computes and tabulates mathematical tables (e.g. polynomial/series values) by the method of finite differences, using repeated addition to avoid multiplication/division, and prints the results — an early automatic, programmable-style calculating engine

Note: the table's first column is the 'Mechanical Calculating Machine' (OCR 'Calculatin'); the second is the 'Difference Machine' (Babbage's Difference Engine).

Unofficial marking guide:

• 1 mark: (a) states the zero-generation machine works by mechanical parts (gears/levers/wheels) rather than electronics, i.e. computation is physical/mechanical.

• 1 mark: (b) correct inventor for the Mechanical Calculating Machine (Pascal/Pascaline, accept Schickard or Leibniz).

• 1 mark: (b) correct function of the Mechanical Calculating Machine (basic arithmetic operations performed mechanically).

• 1 mark: (b) correct inventor for the Difference Machine (Charles Babbage).

• 1 mark: (b) correct function of the Difference Machine (automatic computation/tabulation of tables by finite differences, printing results).

Model confidence: medium.

Model note: The table layout was reconstructed from OCR text; 'Calculatin' is read as 'Calculating'. The question does not name a specific mechanical calculating machine, so Pascal's Pascaline is taken as the standard answer, with Schickard/Leibniz accepted as alternatives. Full marks assume one inventor and one function per column as shown.

QUESTION A2 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A contains the actual/effective memory address of the operand, so the operand is fetched directly from memory location A — there is no intermediate pointer.

(b) How it is used: The instruction is fetched and the CPU decodes the opcode. The address field A is taken as the effective address (EA = A) with no further calculation. The CPU then performs a memory read at address A to bring the operand into a register, and the operation named by the opcode is executed on that operand. In symbolic form: EA = A; Operand = Memory[A]. Because the address is embedded in the instruction, the operand's location is fixed at assembly/compile time and the instruction can only reference that one location unless the instruction is rewritten.

(c) Advantage: It is simple and fast — only one memory access is needed to fetch the operand (no extra indirection or address computation), and no special index/base register is required. Disadvantage: The

© Hak Cipta Universiti Teknologi MARA

address field must be large enough to hold a full memory address, which lengthens the instruction; also the set of addressable locations is limited to the address-field width, and the code is not easily relocatable/position-independent.

Unofficial marking guide:

• 1 mark: correctly names the technique as direct (absolute) addressing.

• 2 marks: explains that the address field A is the effective address (EA = A), the operand is fetched directly from memory[A], and the opcode then operates on it (1 mark for EA = A / address field is the operand address; 1 mark for the fetch-then-execute description).

• 2 marks: 1 mark for a valid advantage (e.g. simple/fast, single memory access, no extra address calculation or index register); 1 mark for a valid disadvantage (e.g. long address field/instruction, limited addressable range, not position-independent/relocatable).

Model confidence: high.

Model note: Figure 2 is inferred from the extracted text (Instruction: Opcode | Address A, with Memory holding the Operand) to depict direct addressing; if the actual figure shows an extra indirection (e.g. address points to a pointer), the answer would instead be indirect addressing. The question's internal reference '9(a)' is treated as a typo for 'a)'.

QUESTION A3 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (13).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three characteristics of a Symmetric Multiprocessor (SMP):

1. Two or more identical processors sharing a single main memory. All CPUs are of the same type and have equal access to a common, globally shared physical memory (typically via a shared bus or interconnection network), so any processor can access any memory location.

2. Symmetric access and equal status — no master/slave relationship. Every processor runs the same operating-system kernel and can execute any task, including kernel/OS code; all processors are peers and share the same I/O devices and system resources.

3. Tightly coupled, single-system image with hardware-supported coherence. The processors cooperate through shared memory under one OS instance, presenting a single system image to users; cache coherence and synchronisation (e.g. locking) are maintained in hardware/OS so shared data stays consistent.

(Other acceptable characteristics: shared single OS, common bus/interconnection, load balancing of processes across CPUs, scalability by adding processors.)

(b) Advantage of SMP in terms of availability:

Because all processors are peers with no single master, the system can continue operating (gracefully degrade) if one processor fails. The remaining processors keep running the OS and the workload, so the failure of a single CPU does not bring the whole system down — the system remains available, merely with reduced processing capacity. This redundancy of processing resources gives higher availability/fault tolerance than a single-processor system, where a CPU failure halts the entire machine.

Unofficial marking guide:

• 1 mark: states that SMP has two or more identical/symmetric processors sharing a single main memory

• 1 mark: states that all processors have equal status/access with no master-slave relationship (same OS, any processor can run any task)

• 1 mark: states tight coupling / single-system image with shared resources and hardware-supported coherence or synchronisation

• 1 mark: identifies that failure of one processor does not stop the system (no single point of failure / graceful degradation)

• 1 mark: explains that remaining processors continue running the OS/workload, so the system stays available with reduced capacity

Model confidence: high.

Model note: Unofficial model answer, not an official marking scheme. The question text has a minor OCR/spacing error ('interms'); interpreted as 'in terms of'. Any three valid SMP characteristics earn the 3 marks in part (a).

QUESTION A4 (5 marks)

Topic: Data representation and number systems

Answer: 378510 = 1110110010012

- *1 mark for identifying the target base.*
- *1 mark for a correct repeated-division or place-value method.*
- *1 mark for the correct digit sequence.*
- *1 mark for the correct base subscript.*
- *1 mark for a complete, consistent workflow.*

QUESTION A5 (10 marks)

Topic: Data representation and number systems

a) Answer: left -60 -> 1 0111100; right -12 -> 1 0001100; -60 - (-12) = -48 -> sign-magnitude 1 0110000

- *1 mark for operand representations.*
- *1 mark for sign handling.*
- *1 mark for magnitude arithmetic.*
- *1 mark for the result sign bit.*
- *1 mark for the final byte.*

b) Answer: left -60 -> 1's 11000011, 2's 11000100; right -12 -> 1's 11110011, 2's 11110100; -60 - (-12) = -48 -> 1's 11001111, 2's 11010000

- *1 mark for the positive operand in binary.*
- *1 mark for the 1's complement of the negative operand.*
- *1 mark for adding 1 to form the 2's complement.*
- *1 mark for the correct 8-bit result.*
- *1 mark for converting the result back to decimal.*

QUESTION A6 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (12).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two functions of an I/O module (4 marks):

Function 1 — Interface/control and timing between the CPU and external devices: The I/O module acts as an intermediary that converts the CPU's internal bus signals (address, data, control) into the timing and signal formats required by the peripheral, and vice versa. It handles the synchronization needed because peripherals operate at different speeds and formats from the CPU and system bus.

Function 2 — Data buffering and error detection: The I/O module temporarily buffers data transferred between the fast CPU/main memory and the slower peripheral, compensating for the speed mismatch, and it performs error detection (e.g., parity/checksum) on transferred data so that faults are reported to the CPU.

(Other acceptable functions: device selection/addressing, status reporting, and control/command decoding — any two functions discussed earn the marks.)

(b) Two categories of external I/O devices with examples (4 marks):

Category 1 — Human-readable (human–machine interface) devices: devices that communicate with the user, e.g., keyboard, monitor/display, printer.

Category 2 — Machine-readable devices: devices that communicate with other equipment/machines, e.g., disk drives, magnetic tape, sensors/actuators, network interface cards.

(The third standard category is communication devices, e.g., modem, network card — not required here.)

(c) I/O techniques (2 marks):

Requires active CPU intervention: Programmed I/O (polling) — the CPU repeatedly checks the device status and performs the transfer itself. (Interrupt-driven I/O is also acceptable as it still involves CPU intervention to service the interrupt.)

Does not require CPU intervention: Direct Memory Access (DMA) — the DMA controller transfers data directly between the device and memory without CPU involvement in each word transfer.

Unofficial marking guide:

- *1 mark: correct identification of first I/O module function (interface/control and timing)*
- *1 mark: discussion/explanation of that first function*
- *1 mark: correct identification of second I/O module function (buffering/error detection)*
- *1 mark: discussion/explanation of that second function*

© Hak Cipta Universiti Teknologi MARA

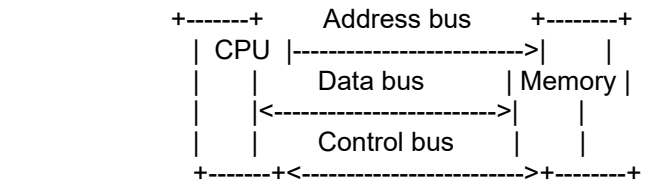
• 1 mark: correct naming of first device category (e.g., human-readable)
• 1 mark: appropriate example device for that category
• 1 mark: correct naming of second device category (e.g., machine-readable)
• 1 mark: appropriate example device for that category
• 1 mark: correct naming of a technique requiring active CPU intervention (programmed I/O/polling or interrupt-driven I/O)
• 1 mark: correct naming of a technique not requiring CPU intervention (DMA)
Model confidence: high.
Model note: Question text has minor OCR/grammar issues ('each categories'); interpreted as 'each category'. The paper states three categories but only two are required; the third (communication devices) is noted for completeness but not marked. Interrupt-driven I/O is accepted as 'active CPU intervention' since the CPU must still service the interrupt, though DMA is the intended 'no CPU intervention' answer.

QUESTION A7 (10 marks)

Topic: Computer architecture and CPU organisation
Source: CSC429 (6).pdf, page(s) 3
Unofficial model answer — generated by an AI model, not an official UTM marking scheme.
(a) Size of the bus: If the word size is 64 bits, the data bus must be 64 bits wide (64 lines) so that a full word can be transferred in one bus cycle. (The address bus width is a separate design parameter and is not fixed by word size.)

(b) Two types of bus: (1) the data bus, which carries data/instructions between components, and (2) the address bus, which carries the address of the memory location or I/O port being accessed. (A control bus is also commonly listed; the two required here are data and address.)

(c) Location of the bus between CPU and memory: The bus sits on the system interconnection between the CPU and main memory, i.e. CPU → system bus (address, data, control lines) → main memory. A simple illustration:



The CPU drives addresses and control signals onto the bus; data flows in both directions between CPU and memory over the shared bus.

(d) System interconnection vs CPU interconnection: System interconnection (the system bus) is the shared communication path linking the major system components — CPU, main memory and I/O — allowing them to exchange data, addresses and control information. CPU interconnection refers to the internal paths/buses within the processor that connect its internal units (registers, ALU, control unit, cache) and carry data and control signals inside the CPU. In short, system interconnection is external and shared among system-level components, whereas CPU interconnection is internal to the processor and connects its functional units.

(e) Drawbacks of bus timing types: (i) Synchronous bus timing — all transfers are governed by a fixed common clock, so the bus must run at the speed of the slowest device; faster devices are held back and the design cannot easily accommodate devices of widely differing speeds. (ii) Asynchronous bus timing — handshaking between master and slave removes the clock-speed limitation, but it requires extra control/handshake lines and more complex control logic, and each transfer incurs handshake overhead, making it slower per transaction than a well-matched synchronous bus.

Unofficial marking guide:
• 1 mark: (a) states the bus (data bus) is 64 bits wide because word size is 64 bits.
• 2 marks: (b) one mark each for correctly naming two bus types (e.g. data bus and address bus).
• 1 mark: (c) correct illustration showing the bus located between CPU and memory with data/address/control lines connecting them.
• 2 marks: (d) one mark for correctly describing system interconnection (shared external path among CPU, memory, I/O) and one mark for CPU interconnection (internal paths within the CPU).
• 4 marks: (e) two marks for synchronous timing drawback (fixed clock, limited by slowest device) and two marks for asynchronous timing drawback (extra handshake lines/complexity/overhead).
Model confidence: high.

© Hak Cipta Universiti Teknologi MARA

Model note: Part (a) is interpreted as asking for the data-bus width implied by a 64-bit word; address-bus width is not determined by word size. Part (b) accepts data/address (or control) buses. The illustration in (c) is a text rendering; an actual diagram would be drawn in the exam.

QUESTION A8 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (15).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Phases in the machine cycle [4 marks]

The machine cycle (instruction cycle) consists of the following phases:

1. Fetch — the next instruction is fetched from main memory into the CPU (into the Instruction Register), with the Program Counter supplying the address.
2. Decode — the Control Unit decodes the fetched instruction to determine the operation and operands.
3. Execute — the required operation is carried out by the ALU and/or other units (arithmetic/logic operation, or effective-address calculation).
4. Memory access / Store (Memory read or write) — if the instruction needs data, the operand is read from memory, or a result is written back to memory.
5. Write-back — the result is written back into a register (or memory).

(Commonly the cycle is summarised as Fetch → Decode → Execute, with memory access and write-back as sub-steps of Execute; either presentation is acceptable provided the phases are clearly listed.)

(b) Function of CPU components in the machine cycle [4 marks]

- Program Counter (PC): holds the address of the next instruction to be fetched; incremented during/after fetch.
- Memory Address Register (MAR): holds the address of the memory location to be accessed (instruction or operand); connected to the address bus.
- Memory Data Register (MDR) / Memory Buffer Register: holds data read from or to be written to memory; connected to the data bus.
- Instruction Register (IR): holds the instruction currently being executed, so the Control Unit can decode it.
- Control Unit (CU): decodes the instruction in the IR and generates the control/timing signals that coordinate all other units through the phases of the cycle.
- Arithmetic Logic Unit (ALU): performs arithmetic and logical operations during the execute phase.
- Registers / Accumulator (general-purpose registers): hold operands and results during execution and write-back.
- Internal buses: carry data and addresses between these units.

(c) If access to main memory is blocked [2 marks]

The fetch phase cannot complete: the CPU cannot read the next instruction from memory into the IR (the MAR/MDR transfer fails), so the Control Unit has no instruction to decode. The machine cycle stalls/halts — the CPU cannot proceed to decode or execute, and the system effectively stops (or waits for the memory to become available). Since instructions and operands both reside in main memory, no useful processing can occur until memory access is restored; the processor may enter a wait/halt state or raise a memory fault/trap.

Unofficial marking guide:

- *1 mark: correctly listing Fetch as a phase*
- *1 mark: correctly listing Decode as a phase*
- *1 mark: correctly listing Execute as a phase*
- *1 mark: correctly listing Memory access/Write-back (store) as a phase*
- *1 mark: correct function of PC/MAR/MDR (address/data handling in fetch)*
- *1 mark: correct function of IR and Control Unit (hold and decode instruction, generate control signals)*
- *1 mark: correct function of ALU (execute arithmetic/logic operations)*
- *1 mark: correct function of registers/accumulator and buses (hold operands/results, transfer data)*
- *1 mark: explains fetch cannot complete / no instruction reaches IR*
- *1 mark: explains cycle stalls/halts and no processing can continue until memory access restored*

Model confidence: high.

Model note: Question text is complete; no figure required. The machine cycle is presented with Fetch/Decode/Execute plus memory-access and write-back sub-phases; some textbooks merge the last two into Execute, so markers should accept either convention. This is an unofficial model answer, not an official marking scheme.

© Hak Cipta Universiti Teknologi MARA

QUESTION A9 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (10).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three types of interfaces to external devices [6 marks]

1. Programmed I/O (polled I/O) interface: The CPU directly controls the transfer and repeatedly checks (polls) the device's status register to see whether it is ready. Data passes through the CPU (e.g., via an accumulator) between the device and memory. It is simple and needs no special interrupt hardware, but it wastes CPU time busy-waiting, so it suits slow or infrequently used devices.

2. Interrupt-driven I/O interface: The device signals the CPU with an interrupt request when it is ready to transfer or has completed a transfer. The CPU suspends its current program, runs an interrupt service routine (ISR) to service the device, then resumes. This frees the CPU from polling, giving better utilisation, though each transfer still involves CPU intervention and interrupt overhead.

3. Direct Memory Access (DMA) interface: A dedicated DMA controller takes over the bus and transfers blocks of data directly between the device and main memory without CPU involvement per word. The CPU programs the controller (start address, count, direction) and is interrupted only once per block. This gives high throughput for fast devices such as disks and network cards, at the cost of extra hardware and bus arbitration (cycle stealing).

(Other acceptable interfaces: memory-mapped I/O vs isolated/port-mapped I/O, serial vs parallel interfaces, or synchronous vs asynchronous interfaces, if described as interfaces to external devices.)

(b) Two types of interrupt [4 marks]

1. Maskable interrupt: An interrupt that can be disabled (masked) by the CPU, typically by clearing a bit in the interrupt-enable register or by the interrupt mask in the status register. It is used for ordinary I/O devices whose service can be delayed; the CPU can ignore it temporarily while executing a critical section. Example: a keyboard or timer interrupt.

2. Non-maskable interrupt (NMI): An interrupt that cannot be disabled by the normal masking mechanism. It is reserved for high-priority, urgent events such as power failure, memory parity error or a hardware fault, and is always recognised so the CPU can take immediate recovery action.

(Alternative acceptable pairing: hardware/external interrupt vs software/internal interrupt, or vectored vs non-vectored interrupt, if each is explained.)

Unofficial marking guide:

- *1 mark: naming the first interface type (e.g., programmed/polled I/O)*
- *1 mark: correct description of how the first interface operates (CPU polls status, data through CPU)*
- *1 mark: naming the second interface type (e.g., interrupt-driven I/O)*
- *1 mark: correct description of how the second interface operates (device raises interrupt, ISR services it)*
- *1 mark: naming the third interface type (e.g., DMA)*
- *1 mark: correct description of how the third interface operates (controller transfers block directly to/from memory, one interrupt per block)*
- *1 mark: naming the first interrupt type (e.g., maskable)*
- *1 mark: explanation of the first interrupt type (can be disabled/masked, used for ordinary I/O)*
- *1 mark: naming the second interrupt type (e.g., non-maskable)*
- *1 mark: explanation of the second interrupt type (cannot be masked, used for urgent/fatal events)*

Model confidence: high.

Model note: Question text is complete; no figure or table is required. The paper's slot label (A9) differs from the question id (A8); assumed the stated 10-mark total (6 + 4) is correct. Alternative valid interface/interrupt pairings are accepted as indicated in the answer.

QUESTION A10 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (21).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Structure A stores data faster. In Structure A the cache hierarchy is L1 → L2 → L3 and the CPU is closest to L1, with L3 sitting between L2 and main memory; in Structure B an extra L4 cache level is inserted between L3 and main memory. Adding a level lengthens the access path and adds another level of tag checking and possible miss handling, so on a miss the CPU must search more levels before

© Hak Cipta Universiti Teknologi MARA

reaching main memory, increasing average memory access time (AMAT = hit time + miss rate × miss penalty). Structure A therefore has the shorter, faster path for storing/retrieving data, assuming comparable technology and sizes. (Note: the exact ordering of blocks in Figure 3 should be read from the figure; the answer follows the level numbering given.)

(b) The fastest cache level is L1 (Level 1 cache). Reason: it is the level closest to the CPU (built into/attached directly to the processor core), so it has the shortest physical and logical access path, the smallest capacity and lowest latency; it is checked first on every memory reference, giving the smallest hit time of all levels.

(c) The main benefit of SRAM is speed: SRAM cells (typically 6-transistor latch-based cells) hold their value as long as power is applied and do not need refresh, so they give very low access latency and can be made to match processor speed. This makes SRAM suitable for on-chip L1/L2/L3 caches, keeping hit time small and reducing the average memory access time; the trade-off is lower density and higher cost per bit than DRAM, which is why caches are small and main memory uses DRAM.

(d) Two write policies for keeping cache and main memory consistent:
1. Write-through: every write to the cache is also written immediately to main memory (usually via a write buffer to avoid stalling). Main memory is always up to date, so consistency is simple, but there is more memory traffic.
2. Write-back (copy-back): writes update only the cache block and set a dirty bit; the block is written back to main memory only when it is replaced (evicted). This reduces memory traffic but requires the dirty bit and a write-back on eviction to maintain consistency.
(Related consistency mechanisms worth a mention: write-allocate vs no-write-allocate on a write miss, and bus snooping/write-invalidate or directory-based coherence protocols in multiprocessors to keep multiple caches consistent.)

Unofficial marking guide:

• 1 mark: correctly identifies Structure A as faster (with the L1–L2–L3 path and no extra L4 level).
• 1 mark: valid justification — shorter access path/fewer levels, lower miss penalty/AMAT, L3 adjacent to main memory in A versus L4 inserted in B.
• 1 mark: identifies L1 as the fastest cache level.
• 1 mark: reason — closest to CPU/shortest access path, smallest and checked first, lowest hit time.
• 1 mark: states SRAM is very fast/low latency (no refresh, latch-based cells).
• 1 mark: links this to cache suitability — matches CPU speed, reduces hit time/AMAT; accepts cost/density trade-off versus DRAM.
• 2 marks: write-through correctly described (write to cache and memory immediately; memory always consistent; write buffer).
• 2 marks: write-back correctly described (dirty bit, write on eviction/replacement; less traffic but needs write-back for consistency).
• Total = 10 marks.
Model confidence: high.
Model note: OCR errors corrected conservatively: 'Identify'→'Identify', 'Vvhat'→'What'. The extracted figure text lists levels without explicit arrows; the answer assumes the standard ordering L1 (closest to CPU) → L2 → L3 → main memory for A, and L1 → L2 → L3 → L4 → main memory for B. If Figure 3 shows a different arrangement, part (a) should be re-read from the figure. This is an unofficial model answer, not a marking scheme.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (18).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption: Figure 2 is interpreted as y = A - ((2C * D) / (B + A^2)). The expression is evaluated as: temp1 = 2*C; temp2 = temp1*D; temp3 = A*A; temp4 = B + temp3; temp5 = temp2 / temp4; y = A - temp5. Registers A, B, C, D are fixed; x, y, z are temporary. Final result must be in y.

One-address program (accumulator-based):

```
LOAD A       ; ACC = A
MUL A        ; ACC = A^2
ADD B        ; ACC = A^2 + B
STOR z       ; z = A^2 + B
LOAD C       ; ACC = C
MUL D        ; ACC = C*D
ADD C        ; ACC = 2*C*D  (since C*D + C*D = 2*C*D)
DIV z        ; ACC = (2*C*D)/(A^2+B)
STOR x       ; x = (2*C*D)/(A^2+B)
LOAD A       ; ACC = A
SUB x        ; ACC = A - x
STOR y       ; y = final answer
```

Two-address program (operand1 = operand1 op operand2):

```
MOV x, C     ; x = C
ADD x, C     ; x = 2C
MUL x, D     ; x = 2C*D
MOV z, A     ; z = A
MUL z, A     ; z = A^2
ADD z, B     ; z = B + A^2
DIV x, z     ; x = (2C*D)/(B + A^2)
MOV y, A     ; y = A
SUB y, x     ; y = A - x
```

Three-address program (dest = src1 op src2):

```
MUL x, C, D   ; x = C*D
ADD x, x, x   ; x = 2*C*D
MUL z, A, A   ; z = A^2
ADD z, B, z   ; z = B + A^2
DIV x, x, z   ; x = (2*C*D)/(B + A^2)
SUB y, A, x   ; y = A - x
```

Unofficial marking guide:

- *1 mark: correct interpretation of Figure 2 as y = A - ((2C*D)/(B + A^2)) and correct order of operations.*
- *6 marks: one-address program – 1 mark for correct use of LOAD/STOR, 1 mark for computing A^2+B, 1 mark for computing 2C*D, 1 mark for division, 1 mark for final subtraction, 1 mark for storing final answer in y.*
- *6 marks: two-address program – 1 mark for MOV to initialise, 1 mark for computing 2C, 1 mark for 2C*D, 1 mark for A^2+B, 1 mark for division, 1 mark for final subtraction into y.*
- *6 marks: three-address program – 1 mark for MUL x,C,D, 1 mark for doubling to 2C*D, 1 mark for MUL z,A,A, 1 mark for ADD z,B,z, 1 mark for DIV x,x,z, 1 mark for SUB y,A,x.*
- *1 mark: correct assembly notation (mnemonics and operand order) consistent with Table 1 for all three programs.*

Model confidence: medium.

*Model note: Figure 2 is not fully legible from the extracted text; the expression is assumed to be y = A - (2C*D)/(B + A^2). If the figure shows a different grouping (e.g., denominator B + A^2 or (B+A)^2), the programs must be adjusted accordingly. Table 1 uses inconsistent punctuation (e.g., ADD (p.q.r)); operand order is taken as dest, src1, src2 for three-address and dest, src for two-address. One-address instructions are assumed to use an implicit accumulator.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA